Quickdress: Design Specification

Milestone 1 · Scoping, Requirements and Architecture · October 13, 2026
Prepared by Marwan Abouelfotuh
Team: Aya Noaman, Malk Metwally, Nada Salem, Marwan Abouelfotuh, Omar Hassan

This document is the companion to the Quickdress Requirements Specification. Requirement IDs (FR-xx, NFR-xx) refer to that document.

Contents

1. High-level architecture
2. Explanation of the architecture
3. Quality attributes
4. Architectural and design patterns
5. Detailed design: class diagram
6. Technical feasibility and risks
7. Conclusion

1 High-level architecture

1.1 Overview

Quickdress is a single iOS app with no backend of its own. Wardrobe data, photos and calendar information stay on the phone. The app makes only two kinds of network call: one to Apple WeatherKit for the forecast and, if the user opts in (FR-22), one to Google's Gemini 3.1 Flash-Lite model through Firebase AI Logic.

The app has three layers, and the presentation layer follows MVVM (Model, View, ViewModel):

- **Presentation layer.** SwiftUI views and their ViewModels. A view draws the screen. Its ViewModel holds the screen's state and calls the layers below.
- **Domain layer.** The app logic in plain Swift. It checks that a clothing item is valid, builds a picture of the day (weather, events, filters) and picks the outfit. It contains no UI code and makes no framework calls.
- **Data and services layer.** A set of protocols, each with an adapter around one Apple framework or external service: SwiftData, the file system, WeatherKit, EventKit, Core Location, UserDefaults and Firebase.

Dependencies point downward only. The layering is relaxed rather than strict, which means a ViewModel may call the data layer directly when there is no logic in between, for example to read the wear log or save filter settings. Sending those calls through a domain class that only forwards them would add code without adding behaviour. Calls never go upward, and no view talks to a framework or the database directly.

1.2 Component diagram

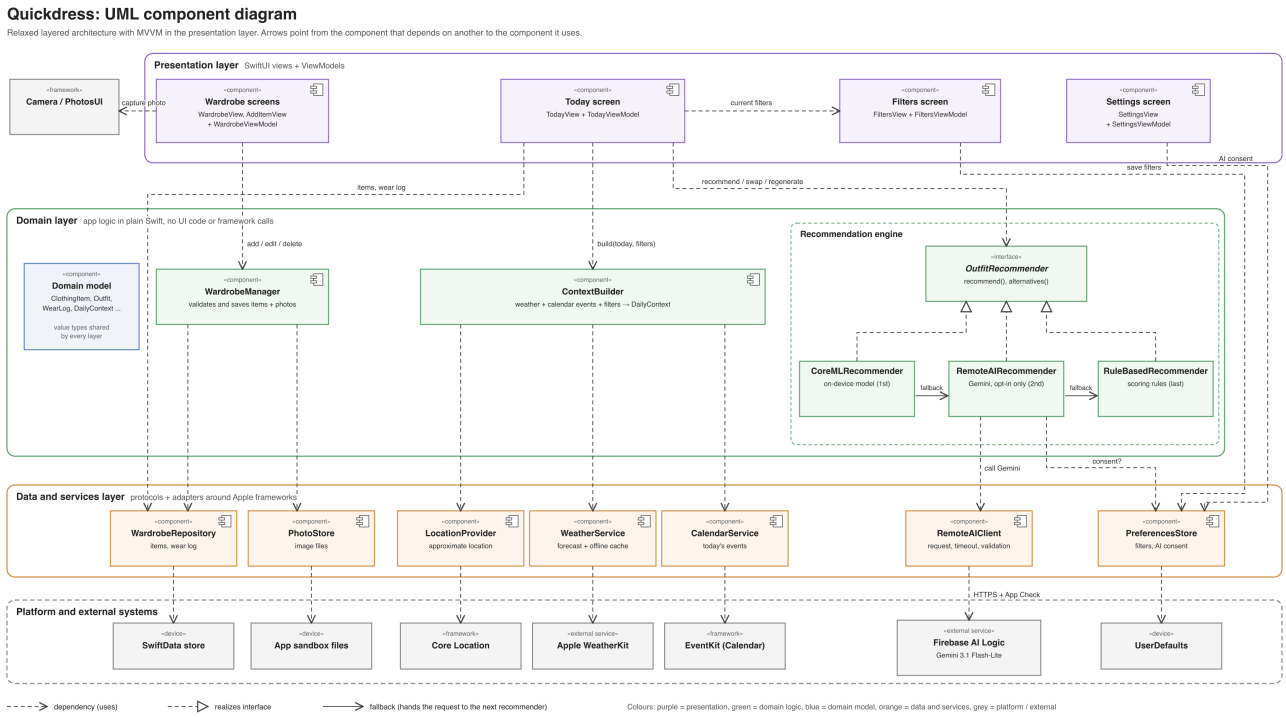


Figure 1. UML component diagram of Quickdress. Colours match the class diagram in Figure 4.

1.3 Components and responsibilities

Table 1 lists every component in Figure 1, what it is responsible for, what it depends on, and which requirements it serves.

Component	Layer	Responsibility	Depends on	Requirements
Today screen TodayView, TodayViewModel	Presentation	Shows the day's outfit. Handles swapping a piece, asking for a new outfit and marking the outfit as worn. Shows a loading state and an offline badge.	ContextBuilder, OutfitRecommender, WardrobeRepository, Filters screen	FR-05, FR-06, FR-07, FR-09, NFR-01
Wardrobe screens WardrobeView, AddItemView, WardrobeViewModel	Presentation	Shows items in a grid with category and colour filters. Adds, edits and deletes items. Opens the camera or the photo library.	WardrobeManager, PhotosUI	FR-01 to FR-04
Filters screen FiltersView, FiltersViewModel	Presentation	Sets transport mode, comfort priority, formality override and the no-repeat period. Hands the current filters to the Today screen.	PreferencesStore	FR-09, FR-10
Settings screen SettingsView, SettingsViewModel	Presentation	Holds the switch for AI suggestions and explains, before the user turns it on, exactly what data is sent.	PreferencesStore	FR-22, NFR-02
WardrobeManager	Domain	Validates item attributes (required fields, ratings from 1 to 5). Saves the photo first and the item second, and deletes both together.	WardrobeRepository, PhotoStore	FR-01 to FR-03, NFR-06
ContextBuilder	Domain	Combines the forecast, calendar events and filters into one DailyContext. Works out the day's formality level. Fills in defaults when a source is missing.	WeatherService, CalendarService, LocationProvider	FR-08, FR-10, FR-11, NFR-03, NFR-07
OutfitRecommender (interface)	Domain	The one contract for producing an outfit: `recommend()` and `alternatives(for:slot:)`.	None	NFR-08
CoreMLRecommender	Domain	Scores combinations with an on-device Core ML model. Passes the request on when no model is bundled or its confidence is low.	Core ML model file, next recommender	FR-05, NFR-02
RemoteAIRecommender	Domain	Asks Gemini for an outfit when the user has opted in and the phone is online. Checks the reply against the wardrobe. Otherwise passes the request on.	RemoteAIClient, PreferencesStore, next recommender	FR-05, FR-22, NFR-02
RuleBasedRecommender	Domain	Scores items with fixed rules: warmth against temperature, formality against the day, a colour compatibility table, transport and comfort, and recent wear. Always returns an outfit if the wardrobe has a top, a bottom and shoes.	None	FR-05 to FR-10, NFR-03, NFR-05
Domain model	Domain	Types shared by every layer: ClothingItem, Outfit, WearLog, DailyContext, DailyFilters, Forecast, CalendarEvent and the enumerations.	None	FR-02
WardrobeRepository	Data	Stores and fetches items and the wear log.	SwiftData	FR-02, FR-03, FR-09, NFR-06

Component	Layer	Responsibility	Depends on	Requirements
PhotoStore	Data	Saves, loads and deletes photo files in the app's sandbox. The database keeps only a reference to each file.	File system	FR-01, NFR-06
WeatherService	Data	Fetches the forecast for a date and place, and caches the last result for offline use.	WeatherKit	FR-08, NFR-03
CalendarService	Data	Asks for calendar access the first time it is needed and reads the day's events.	EventKit	FR-11, NFR-07
LocationProvider	Data	Gets an approximate location for the forecast.	Core Location	FR-08, NFR-07
PreferencesStore	Data	Keeps the saved filters and the AI consent setting.	UserDefaults	FR-09, FR-10, NFR-02
RemoteAIClient	Data	Builds the anonymised request, calls Gemini through Firebase AI Logic with a timeout, and parses the JSON reply.	Firebase AI Logic	FR-22, NFR-02, NFR-08

Table 1. Components, responsibilities and the requirements they serve.

1.4 Relationships and interfaces

The components follow four rules:

1. **Dependencies point down.** A component may use components in its own layer or in a lower one, never in a higher one.
2. **Every service sits behind a protocol.** The domain and presentation layers depend on `WardrobeRepository`, `WeatherService` and so on, never on the concrete adapters (`SwiftDataWardrobeRepository`, `WeatherKitService`, `EventKitCalendarService`). That is why Figure 1 shows only the protocols.
3. **Objects are wired together in one place.** The app's entry point (`QuickdressApp`) creates the adapters, the recommender chain and the managers, then passes them to the ViewModels through their initialisers. Tests and SwiftUI previews pass fakes in the same way.
4. **The domain model is shared.** Items, outfits and the other model types are used by all three layers. They are marked `@Model` so that SwiftData can store them directly. This is the only place where the domain layer knows about a framework. We accept it because a second set of storage types would double the model code for little gain. All queries and the `ModelContext` stay inside `SwiftDataWardrobeRepository`.

Table 2 lists the interfaces between components.

Interface	Provided by	Used by	Main operations
OutfitRecommender	CoreMLRecommender, RemoteAIRecommender, RuleBasedRecommender	TodayViewModel	`recommend(items, wearLog, context, filters) -> Outfit?` `alternatives(for:slot:) -> [ClothingItem]`
WardrobeRepository	SwiftDataWardrobeRepository	WardrobeManager, TodayViewModel	`fetchItems()`, `save(_:)`, `delete(_:)`, `fetchWearLog(since:)`, `logWorn(_:on:)`
PhotoStore	File-based PhotoStore	WardrobeManager	`save(image) -> ref`, `load(ref)`, `delete(ref)`
WeatherService	WeatherKitService (OpenMeteoWeatherService as a backup, see R1)	ContextBuilder	`forecast(for:at:)`, `cachedForecast()`
CalendarService	EventKitCalendarService	ContextBuilder	`requestAccess()`, `events(on:)`

Interface	Provided by	Used by	Main operations
LocationProvider	CoreLocationProvider	ContextBuilder	`currentLocation() -> Location?`
PreferencesStore	UserDefaultsPreferences Store	FiltersViewModel, SettingsViewModel, RemoteAIRecommender	`filters`, `aiConsent`
RemoteAIClient	FirebaseGeminiClient	RemoteAIRecommender	`suggestOutfit(request) async throws -> OutfitSuggestion`

Table 2. Interfaces between components.

2 Explanation of the architecture

This section follows two requests through the system, then explains the recommendation engine and the alternatives we rejected. Figure 2, on the next page, shows the first request.

2.1 Viewing today's outfit

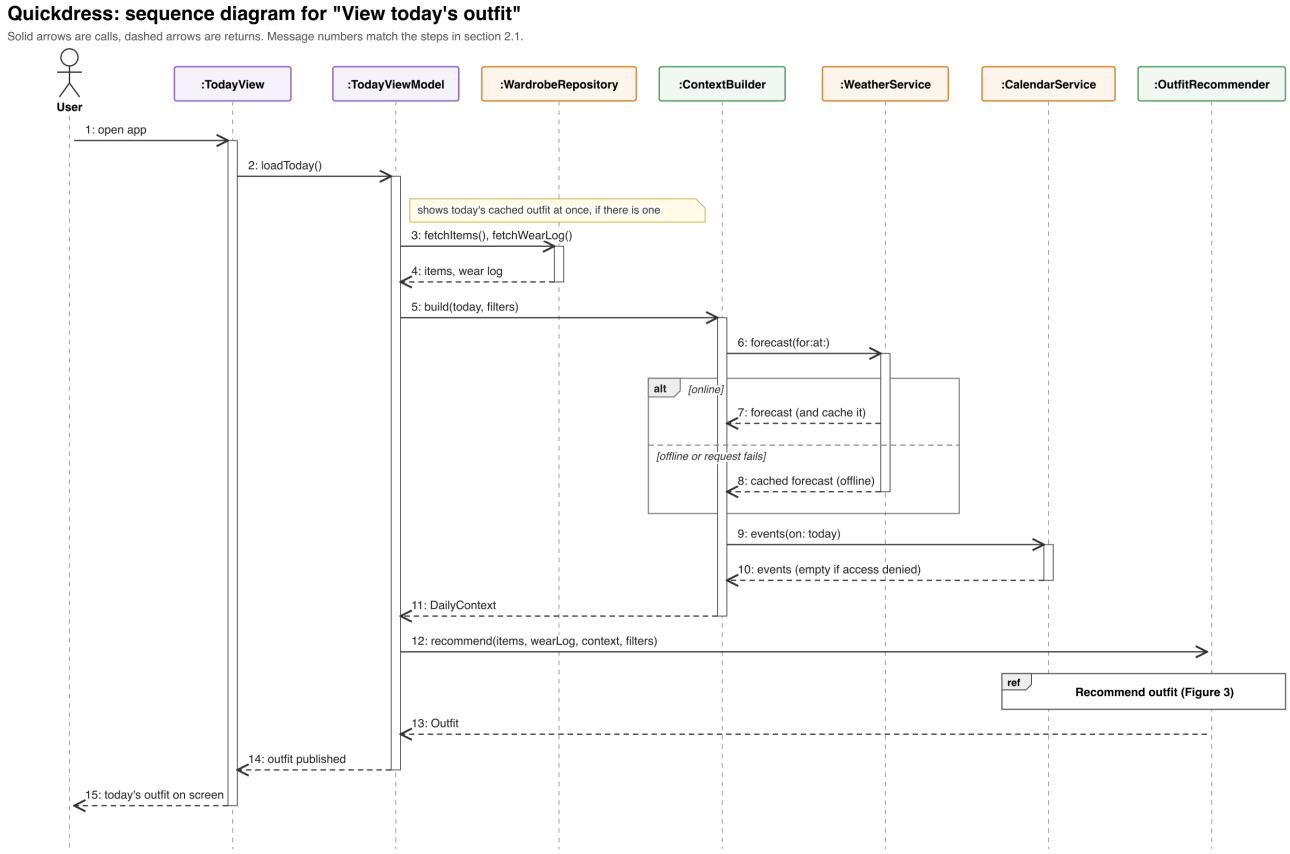


Figure 2. Sequence diagram for the "View today's outfit" use case (FR-05, FR-08, FR-11, NFR-01). The recommender step is shown in Figure 3.

Figure 2 shows the main use case. The numbers in brackets are the message numbers in the figure.

1. The user opens the app and `TodayViewModel.loadToday()` runs (1, 2). If an outfit was already generated today, it is shown from the cache straight away, so the screen is never blank (NFR-01).
2. The ViewModel loads the items and the recent wear log from `WardrobeRepository` (3, 4).
3. It asks `ContextBuilder` for a `DailyContext`, passing today's date and the filters from the Filters screen (5).
4. `ContextBuilder` gets an approximate location from `LocationProvider` and the forecast from `WeatherService` (6, 7). If the network is down or the request fails, `WeatherService` returns the last cached forecast instead and the context is marked offline (8). The Today screen then shows a small badge. If location permission is denied, the context uses a default temperature range and the user is told the outfit ignores the weather.
5. `ContextBuilder` reads today's events from `CalendarService` (9, 10). If calendar access is denied, the list is empty and formality comes only from the filters (NFR-07).
6. It works out the formality level. An event the user tagged as formal, or one whose title contains a keyword such as "presentation" or "interview", raises the level. A formality override set in the filters wins over both. The finished `DailyContext` goes back to the ViewModel (11).
7. The ViewModel calls `recommend()` on the `OutfitRecommender` it was given (12, 13). That object is the first link of the fallback chain, so the ViewModel never knows which recommender produced the outfit. Figure 3 in section 2.3 shows what happens inside the chain.
8. The outfit is published (14). SwiftUI redraws the screen, and the outfit is cached for the rest of the day (15).

Swapping a piece (FR-06) calls `alternatives(for:slot:)` on the same chain and replaces one slot with `Outfit.replacing(slot:with:)`. Asking for a new outfit (FR-07) calls `recommend()` again, excluding the current outfit. Marking an outfit as worn (FR-09) calls `logWorn(_:on:)` on the repository, and every recommender then skips that combination for the no-repeat period.

2.2 Adding a clothing item

The Wardrobe screen opens the camera or the photo library. Permission is asked for only at that moment (NFR-07). The user sets category, colour, warmth, formality, comfort and seasons, then taps Save. `WardrobeManager.addItem` validates the attributes and asks `PhotoStore` to write the image file, which returns a reference. Only then does it save the item, with that reference, through `WardrobeRepository`. If saving the item fails, the manager deletes the photo it has just written, so no orphan files are left behind (NFR-06). Deleting an item runs in the opposite order. If the deleted item is part of today's cached outfit, the outfit is regenerated.

2.3 The recommendation engine and its fallback chain

The engine has three recommenders. All three implement `OutfitRecommender` and are linked in a fixed order. Each one holds a reference to the next (the `fallback` property in the class diagram), and passes the request on when it cannot answer. Figure 3 shows the chain at work.

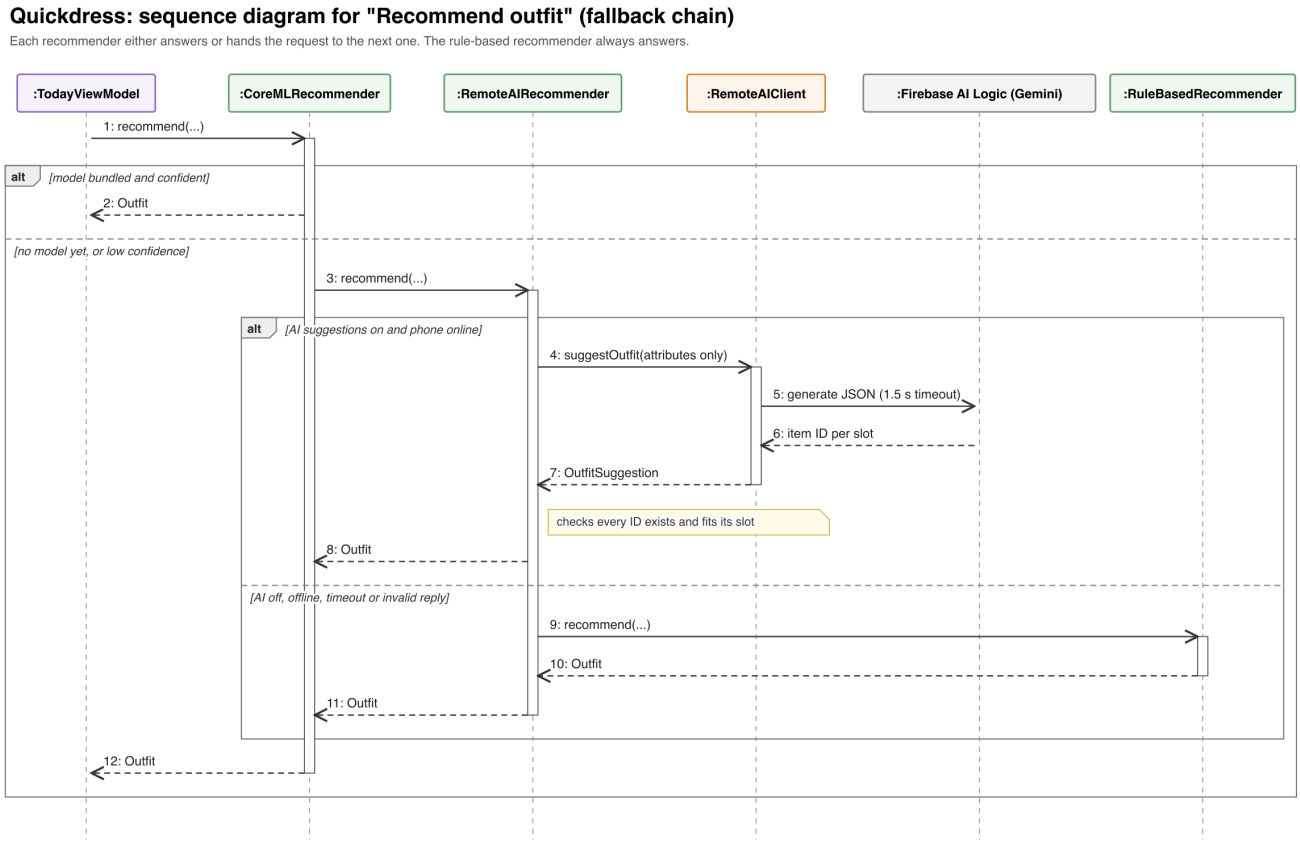


Figure 3. Sequence diagram for "Recommend outfit", the fallback chain referenced in Figure 2 (FR-05, FR-22, NFR-02, NFR-03, NFR-08).

Recommender	Used when	Data leaving the phone	Works offline	Built in
1. CoreMLRecommender	A trained model is bundled with the app and its confidence is above a threshold.	None	Yes	After the rule-based version is stable (stretch goal)
2. RemoteAIRecommender	The Core ML step passed the request on, the user has turned on AI suggestions in Settings, and the phone is online.	Anonymised attributes only (see below)	No	Sprint 3, if there is capacity (FR-22)
3. RuleBasedRecommender	Always, as the last link.	None	Yes	Sprint 2

Table 3. The three recommenders in fallback order.

Until a Core ML model exists, `CoreMLRecommender` forwards every request, so the chain already works in Sprint 2 when only the rule-based scorer is built. The last link needs nothing from outside the phone, which means the app can always produce an outfit. This is what NFR-03 asks for.

The rule-based scorer first scores each item on its own for its slot (warmth, formality, comfort, season, recent wear), keeps the best five per slot, and only then scores the combinations for colour compatibility. That is at most 5 × 5 × 5 × 5 = 625 combinations however large the wardrobe is, which keeps generation well under the one-second limit in NFR-05.

What is sent to Gemini

When the remote step runs, `RemoteAIClient` sends:

- for each item: its ID, category, colour, warmth, formality, comfort and seasons;
- for the day: the temperature range, the chance of rain, the formality level, the transport mode and the comfort priority;
- the IDs of items worn during the no-repeat period.

It never sends photos, item names, calendar event titles or the user's location. Gemini 3.1 Flash-Lite is asked for structured JSON output that names one item ID per slot. `RemoteAIClient` rejects any reply that is not valid JSON, names an ID that is not in the wardrobe, or puts an item in the wrong slot. In any of those cases, or if no reply arrives within 1.5 seconds, the request goes on to the rule-based recommender. The model name is stored in Firebase Remote Config, as Firebase recommends, so it can be changed without an app update.

The AI switch is off by default. Before the user turns it on, the Settings screen explains in one sentence what is sent. This is the explicit consent that NFR-02 requires.

Why Firebase AI Logic instead of calling the Gemini API directly

Calling the Gemini API directly would mean shipping an API key inside the app, and anything inside an app bundle can be extracted. Firebase AI Logic keeps the key on Google's side and uses App Check (App Attest on iOS) to accept requests only from genuine copies of our app. We still run no server of our own and have no user accounts, as the Requirements Specification commits to. Firebase is a Google-hosted service, used only by the opt-in step (FR-22).

2.4 Alternatives considered

Alternative	Why we did not choose it
Model, View, Controller (MVC)	SwiftUI has no view controllers, so in practice the logic ends up inside the views, where it is hard to test. MVVM fits SwiftUI's data binding (`@Observable`) directly.

Alternative	Why we did not choose it
Client and server with our own backend	It would allow syncing and a server-side model, but it breaks on-device privacy (NFR-02), needs hosting and user accounts, and adds a second codebase. iCloud sync is already out of scope this term (FR-21).
Strict (closed) layering	Every ViewModel call to the data layer would need a domain method that only forwards it. That adds code with no behaviour behind it.
Calling the Gemini API straight from the app	The API key would have to ship inside the app, where it can be extracted and abused.

3 Quality attributes

Table 4 shows how the architecture meets each non-functional requirement in scope, and how we will check it.

Requirement	How the architecture addresses it	How we will check it
NFR-01 Performance: outfit visible within 2 s	The Today screen shows the cached outfit at once. Recommendation runs in an async task off the main thread. The remote call has a 1.5 s timeout and is skipped when offline.	Measure launch to outfit on an iPhone 12 with a 200-item test wardrobe, using Instruments.
NFR-02 Privacy: data stays on the device	All storage is local (SwiftData, sandbox files, UserDefaults). `RemoteAIClient` is the only component that sends wardrobe data, it sends attributes only, and `RemoteAIRecommender` checks consent before every call. The location goes to WeatherKit only after the user grants location permission.	Inspect network traffic with a proxy while consent is off: only weather requests should appear. Code review checks that no other component builds outgoing requests.
NFR-03 Availability: works offline	The rule-based recommender is always the last link and needs no network. `WeatherService` caches the last forecast, and the context is marked offline.	Run the app in airplane mode, then browse the wardrobe and generate an outfit.
NFR-04 Usability: first outfit within 5 minutes	Two tabs (Today, Wardrobe) and filters in a sheet. When the wardrobe is missing a top, a bottom or shoes, the Today screen shows an empty state with an Add item button instead of an error.	Time three to five people who have never seen the app.
NFR-05 Performance: under 1 s for 200 items	Per-slot shortlist before combining (section 2.3). The scorer is a pure Swift function with no input or output, so it can be timed in isolation.	XCTest performance test (`measure`) on 200 generated items.
NFR-06 Reliability: no data lost	SwiftData persistent store. Photos are saved before the item and removed if saving fails. Files live in Application Support, not in the caches folder that iOS may clear.	Force-quit during a save. Install a new build over an old one and check that all items remain.
NFR-07 Security: permissions on first use	Each adapter asks for its own permission the first time it is needed, with a purpose string in Info.plist. Every service returns a safe default when access is denied, and `ContextBuilder` handles missing data.	Deny each permission in turn and check that the app still works.
NFR-08 Maintainability: swappable recommender	The `OutfitRecommender` protocol. TodayViewModel holds only the protocol, and the chain is assembled in one place, so adding or removing a recommender changes one line.	UI tests run with a fake recommender, with no change to any view.

Table 4. Non-functional requirements and the architectural decisions that meet them.

NFR-09 to NFR-11 are not formally tested this term, but the design supports them. Standard SwiftUI controls work with VoiceOver and Dynamic Type, and we will add labels to item photos. Strings go in a String Catalog so Arabic can be added later, and SwiftUI mirrors layouts for right-to-left languages. The deployment target is iOS 17, which SwiftData and `@Observable` need anyway.

4 Architectural and design patterns

Two architectural patterns shape the whole app (layers and MVVM). Five design patterns shape individual parts of it. For each one, the table gives what the pattern is, where we use it, and why it suits Quickdress.

4.1 Architectural patterns

Layered architecture

What it is	Components are grouped into layers by level of abstraction. Each layer uses only the layers below it.
Where it is used	Presentation, domain, and data and services (Figure 1). The layering is relaxed: a ViewModel may skip the domain layer when there is no logic in between.
Why it fits	It keeps Apple framework code at the bottom, where it can be replaced (for example WeatherKit by Open-Meteo) without touching the logic. The domain layer can be unit tested without a device. It also matches the order we build in: each sprint adds a vertical slice through all three layers.

Model, View, ViewModel (MVVM)

What it is	The View displays state, the ViewModel holds that state and the actions on it, and the Model is the data. The View observes the ViewModel and redraws when the state changes (an instance of the Observer pattern).
Where it is used	Every screen: TodayView with TodayViewModel, WardrobeView with WardrobeViewModel, and so on. ViewModels are `@Observable` classes.
Why it fits	SwiftUI is built around this kind of binding. ViewModels can be tested without any UI, and the people building screens and the people building logic can work in parallel.

4.2 Design patterns

Strategy

What it is	A family of interchangeable algorithms behind one interface, so the caller can use any of them without knowing which.
Where it is used	The `OutfitRecommender` protocol and its three implementations.
Why it fits	NFR-08 asks for this directly. We can ship the rule-based recommender in Sprint 2 and add Core ML or Gemini later without changing TodayViewModel.

Chain of Responsibility

What it is	A request is passed along a chain of handlers until one of them handles it.
Where it is used	CoreMLRecommender, then RemoteAIRecommender, then RuleBasedRecommender, linked through the `fallback` property. Each link is also a Strategy implementation, so the whole chain looks like one recommender to the caller.
Why it fits	Each recommender decides for itself whether it can answer, so the fallback logic is not one large if-else in the ViewModel. The order is set in one place, and the last link always answers (NFR-03).

Repository

What it is	An object that behaves like a collection of domain objects and hides how they are stored.
Where it is used	The `WardrobeRepository` protocol and `SwiftDataWardrobeRepository`.
Why it fits	All queries and the SwiftData `ModelContext` live in one class. Tests and SwiftUI previews use an in-memory fake. Storage could later move to iCloud (FR-21) without changes elsewhere.

Adapter

What it is	A class that wraps an existing interface so it matches the interface a client expects.
Where it is used	`WeatherKitService`, `EventKitCalendarService`, `CoreLocationProvider` and `FirebaseGeminiClient` wrap Apple and Google SDKs behind our own protocols and convert their types into our `Forecast` and `CalendarEvent`.
Why it fits	SDK types do not leak into the domain layer. If WeatherKit is not available (risk R1), we write one new adapter for Open-Meteo and nothing else changes.

Dependency injection

What it is	Objects receive the things they depend on from outside instead of creating them.
Where it is used	`QuickdressApp` creates the adapters, the recommender chain and the managers, and passes them to the ViewModels through their initialisers.
Why it fits	It is what lets tests and previews use fakes, and it turns the order of the recommender chain into a configuration decision rather than code spread across classes.

5 Detailed design: class diagram

Quickdress: UML class diagram

Domain models (blue), recommendation and wardrobe logic (green), data access protocols and adapters (orange), ViewModels (purple), enumerations (grey). Swift types.

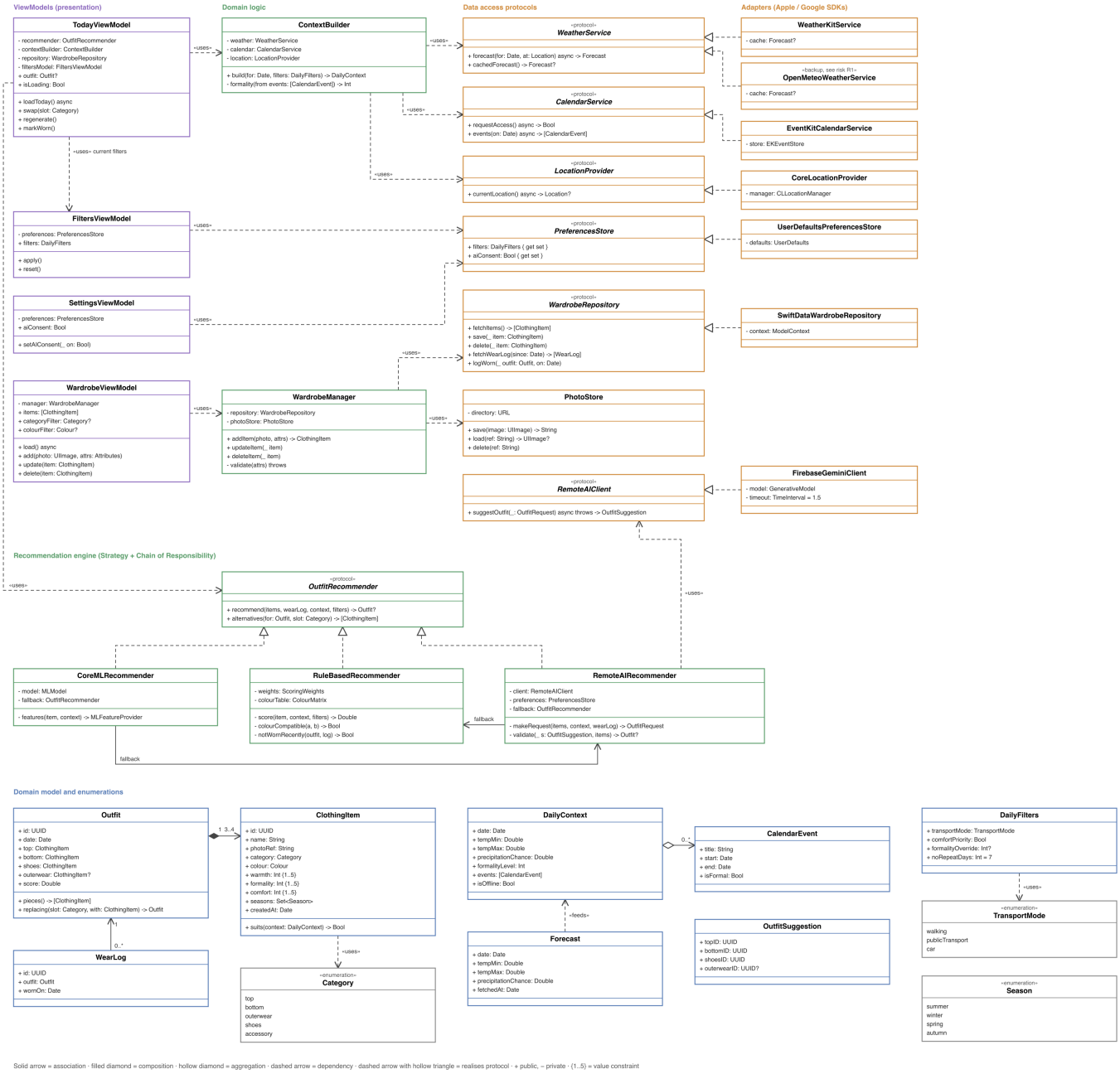


Figure 4. UML class diagram (Swift types). The components in Figure 1 are made up of these classes.

Table 5 maps each component in Figure 1 to its classes in Figure 4.

Component	Classes in Figure 4
Today screen	TodayViewModel
Wardrobe screens	WardrobeViewModel
Filters screen	FiltersViewModel
Settings screen	SettingsViewModel
WardrobeManager	WardrobeManager
ContextBuilder	ContextBuilder
Recommendation engine	OutfitRecommender, CoreMLRecommender, RemoteAIRecommender, RuleBasedRecommender
Domain model	ClothingItem, Outfit, WearLog, DailyContext, Forecast, CalendarEvent, DailyFilters, OutfitSuggestion, Category, Season, TransportMode
WardrobeRepository	WardrobeRepository, SwiftDataWardrobeRepository
PhotoStore	PhotoStore
WeatherService	WeatherService, WeatherKitService, OpenMeteoWeatherService (backup, see R1)
CalendarService	CalendarService, EventKitCalendarService
LocationProvider	LocationProvider, CoreLocationProvider
PreferencesStore	PreferencesStore, UserDefaultsPreferencesStore
RemoteAIClient	RemoteAIClient, FirebaseGeminiClient

Table 5. Mapping from components to classes.

Every protocol in the data layer has its adapter drawn next to it, joined by a realisation arrow. Dependencies that a class holds as a typed attribute, such as `TodayViewModel.repository`, are listed in the class box; only the main ones are drawn as arrows to keep the diagram readable.

6 Technical feasibility and risks

6.1 Feasibility

Every in-scope feature uses well-documented first-party Apple frameworks: SwiftUI, SwiftData, PhotosUI, WeatherKit, EventKit and Core Location. The only third-party SDK is Firebase, and only the optional Gemini step uses it. The hardest algorithm, the recommender, starts as rule-based scoring in ordinary Swift. Core ML and Gemini are optional links in the chain, and the app is complete without them.

The architecture also sets the build order. Each sprint delivers a working vertical slice through all three layers:

Sprint	Components built	Result
Sprint 1	Wardrobe screens, WardrobeManager, WardrobeRepository, PhotoStore, domain model	A working digital wardrobe (FR-01 to FR-04)
Sprint 2	Today screen, ContextBuilder, WeatherService, LocationProvider, RuleBasedRecommender, with CoreMLRecommender forwarding every request	A daily outfit matched to the weather (FR-05 to FR-08)

Sprint	Components built	Result
Sprint 3	CalendarService, Filters screen, PreferencesStore, wear log. If there is capacity: FR-12, FR-13 and FR-22 (Settings screen, RemoteAIRecommender, RemoteAIClient)	Calendar, filters and no repeats (FR-09 to FR-11)

Table 6. Build order by sprint.

6.2 Risks

ID	Risk	Likelihood	Impact	Mitigation
R1	WeatherKit needs a paid Apple Developer Program membership ($99 a year). Without one, the app also has to be reinstalled on test phones every 7 days.	High	High: blocks FR-08	Find out in week 1 whether the university or a team member has a membership. If not, write `OpenMeteoWeatherService` (free for non-commercial use, no key) behind the same `WeatherService` protocol.
R2	Xcode runs only on macOS. A team member without a Mac cannot build or run the app.	Medium	High	Check in week 1. Pair those members with Mac owners, or give them work that does not need Xcode: the colour compatibility table and scoring weights, test plans, Firebase setup and documentation.
R3	The team is learning Swift and SwiftUI during the project.	High	Medium	Sprint 1 is plain create, read, update and delete work. Every pull request is reviewed. The layers let each person work on one part at a time.
R4	There is no training data for the Core ML model.	High	Low	The rule-based recommender is the product. Core ML is only attempted if we can collect real usage data, which probably means after this term.
R5	Gemini replies may be wrong, slow or not valid JSON.	Medium	Low	Structured output, validation against the wardrobe, a 1.5 s timeout and the rule-based fallback.
R6	Gemini cost, quota or model names change.	Low	Low	Flash-Lite is Google's lowest-cost model in the family. The model name is in Remote Config. Usage is opt-in and limited to one call per generated outfit.
R7	Detecting formality from calendar titles by keyword is unreliable (vague titles such as "Meeting", or titles in Arabic).	High	Medium	The user can tag an event as formal. The formality override in the filters always wins. The keyword list is kept in one editable place.
R8	SwiftData is fairly new and has known bugs, especially with schema changes.	Medium	Medium	All access goes through `WardrobeRepository`, so we could move to Core Data. We keep the schema small and test upgrades (NFR-06).
R9	Automatic category and colour suggestion (FR-13) may be inaccurate.	Medium	Low	It is a stretch goal, and the user always confirms or corrects the suggestion.
R10	The iOS Simulator has no camera.	Certain	Low	Use photo library import in the Simulator and test the camera on a real phone.

Table 7. Technical risks.

R1 and R2 need answers in the first week, because both could stop part of the team from building the app at all.

7 Conclusion

Quickdress is split into presentation, domain, and data and services layers, with MVVM on every screen. Each Apple framework and external service sits behind a protocol, and the recommender sits behind a single interface. Behind that interface is a fallback chain whose last link needs no network and sends no data, which is how the design meets the offline and privacy requirements while still allowing an on-device model and an opt-in Gemini step. Table 4 traces each of NFR-01 to NFR-08 to the decision that meets it, and Table 6 shows that the build order fits the three sprints in the Requirements Specification.